

KURUMSAL YAZILIMLAR

Kurumsal Yazılım İçin Gereklilikler

Kurumsal bir yazılım sisteminin inşası için aşağıdakiler sırasıyla yapılır;

- **İşlevsel model** (**functional model**) çıkarılır. Kullanıcılara **ne** (**what**) yapmak istedikleri sorulur ve use-case diyagramları oluşturulur.
- **Yapısal model** (**structural model**) çıkarılır. İşlemlerin **kimler** (**who**) tarafından yapılacağı belirlenir ve bu kapsamda sınıf diyagramları oluşturulur.
- **Davranış modeli** (**behavioral model**) oluşturulur. İşlemlerin kim tarafından **nasıl** (**how**) ve hangi sırada yapılacağı belirlenir ve silsile diyagramları oluşturulur.
- **Mimari modelleme** (**architectural model**) oluşturulur. Bu modelde nesnelerin nasıl ve nerede (**where**) bir araya geleceği üzerine durulur ve yayılma ve paket diyagramları oluşturulur. Bu modelde iki gereklilik detaylı incelenir. Birincisi iş süreçlerini ya da kullanıcı isteklerini yerine getiren işlevsel gerekliliklerdir. İkincisi ise yazılımla birlikte ortaya çıkan performans, **güvenlik** (**security**), **günlük** (**logging**) ve benzeri gerekliliklerdir.

Kurumların iş süreçlerinin de dâhil edildiği büyük yazılımlar **kurumsal uygulamalar** (**enterprise application**) olarak adlandırılır. Adından da anlaşılacağı üzere, kurumsal yazılımlar, kurumun iş süreçlerini elektronik ortama taşır. Kurumsal yazılım geliştirmek için aşağıda belirtilen üç şeye sahip olmamız gerekir:

1. **Takım** (**Team**): Analizci, testçi ve programcı gibi elemanlardan oluşan uygulama geliştirecek olan yazılımcı grubu.
2. **Yazılım Geliştirme Yaşam Döngüsü** (**Software Development Life Cycle-SDLC**): Kurumsal yazılım geliştirme için kullanılan modeller; **Şelale modeli**(**waterfall model**), **Rational Unified Process** (**RUP**), **Çevik model** (**agile model**) gibi modeller ya da kendimize has yazılım geliştirme modelimiz.
3. **Teknoloji** (**Technology**): Yazılımın, sahip olacağı teknoloji, kurumsal yazılım geliştirme için gerekli bileşenleri programcıya sunarlar. Bu teknolojiler, yazılım geliştirme sırasında gerekli olan bileşenleri, bir mağazadan alıp kullanır gibi programcıya sunarlar. Böylece programcının iş süreci üzerine yoğunlaşarak daha hızlı ve doğru bir çözüm üretmesini sağlarlar. Jakarta EE (Java Enterprise Edition) ve .NET Framework en çok kullanıldığı teknolojilerdir.

Yazılım geliştirme sırasında bu üç şeyi çok iyi seçmemiz halinde başarılı bir kurumsal yazılım ortaya çıkar. Burada özellikle teknoloji üzerinde durmak gerekir. Jakarta EE ve .NET gibi iki büyük teknolojinin programcıya sunduğu bileşenler arkasında birçok sunucu çalışır. Bunların bazıları aşağıda verilmiştir;

- Kurumsal yazılım geliştirmede kullanılan modüler bileşenleri sunan sunucular: Jakarta Enterprise Beans, Component Object Model-COM, COM+ gibi teknolojileri sağlayan .NET Bileşenleri
- Kullanıcı, bilgisayar, yazıcı ve benzeri öğeleri içeren kurumsal dizin sunucuları: Java Naming and Directory Interface (JNDI), Microsoft Active Directory
- Kurumsal Uygulamada bileşenlerin birbirine **ileti** (**message**) gönderip almalarını sağlayan servisler: Jakarta Messaging API, Microsoft Message Queue (MSMQ)
- Web Sunucuları üzerinde bileşenlerin çalışmasını sağlayan servisler: Jakarta Server Pages (JSP), ASP.NET Core (Active Server Pages for .NET)
- Mevcut kurumsal uygulamaları birbirine bağlayan servisler: Jakarta Connectors, Java Database Connectivity (JDBC), Java XML, Jakarta Transactions, ADO.NET (Active Data Objects for .NET), WCF (Windows Communication Foundation)

JEE ile .NET teknolojileri arasında seçim yapmak için kurumun sahip olduğu donanım altyapısı ve yukarıda belirtilen servisleri sağlayan yazılım kaynakları irdelenmelidir.

Tüm projelerde olduğu gibi kurumlarda geliştirilen uygulamalar da proje olarak ele alınır ve yönetilirler. Proje yönetimi, belirli bir hedefe ulaşmak için zaman, bütçe, kaynaklar ve kalite kısıtları altında yürütülen işleri planlama, yürütme, izleme ve tamamlama sürecidir. Kısaca doğru işi, doğru şekilde, doğru zamanda ve doğru kaynaklarla yapmak demektir.

Proje yönetimi genellikle beş ana aşamada yürütülür:

1. **Başlatma (initiation)**: Projenin amacı, kapsamı ve paydaşları belirlenir. Proje yöneticisi atanır ve gerekirse fizibilite çalışması yapılır. Proje yöneticisi, proje planı hazırlamak ve güncel tutar, takımı yönetir, iletişimi sağlar, riskleri öngörür ve önlem alır, paydaşlarla iletişimi sürdürür ve teslimatları zamanında gerçekleştirir.
2. **Planlama (planning)**: Proje hedefleri, teslimatlar belirlenir. **Zaman çizelgesi (Gantt Chart)** ve alt kırılımlar oluşturulur. Kaynak planlaması, risk yönetimi, bütçe, iletişim planı hazırlanır.
3. **Yürütme (execution)**: Takım kurulur, görevler dağıtılır. İşler gerçekleştirilir, yazılım geliştirme, inşaa gibi ana faaliyetler yapılır. İletişim ve koordinasyon sağlanır.
4. **İzleme ve Kontrol (monitoring and controlling)**: Proje ilerlemesi takip edilir. Gecikmeler, sapmalar analiz edilir. Riskler yönetilir, gerekirse planlar revize edilir.
5. **Kapanış (closing)**: Proje sonuçlandırılır. Teslimatlar yapılır. Geri bildirim alınır, raporlama yapılır, dersler çıkarılır.

Proje yönetimini yapmak için Microsoft Project, Jira, Trello, Asana, ClickUp, Notion, Monday.com, Smartsheet ve Wrike gibi araçlar kullanılır.

Bir yazılım projesini yönetmek için yukarıdaki yönetim aşamalarını içerecek **şekilde yazılım geliştirme yaşam döngüsü (Software Development Life Cycle-SDLC)** ile yapılır. Yazılım geliştirme modeli olarak da adlandırılan SDLC, kaliteli yazılım tasarlamak, geliştirmek ve test etmek için kullanılan yapılandırılmış bir süreçtir. SDLC, bir yazılım mühendisi veya geliştiricisi tarafından çeşitli aşamalarda gerçekleştirilecek görevi/görevleri belirtir. Son ürünün müşterinin beklentilerini karşılayabilmesini ve tanımlanan bütçeye uymasını sağlar³⁷.

Burada **yazılım mühendisliği (software engineering)** devreye girer. Yazılım mühendisliği, yazılım sistemlerinin planlanması, tasarımı, geliştirilmesi, test edilmesi, dağıtımı ve bakımı ile ilgilenir.

Yazılım mühendisi;

- Yazılım geliştirme veya kısaca programlama yapar,
- Yazılım mimarisi belirler,
- Yazılım testini yapar ve kalitesini belirler,
- Yazılım proje yönetimini yapar,
- Yazılım yaşam döngüsü sürecini (SDLC) yürütür,
- **Geliştirme ve işletim (Developer Operations-DevOps)** süreci ile **sürekli entegrasyon/sürekli dağıtım (Continuous Integration/Continuous Delivery-CI/CD)** süreçlerini yürütür.

Yazılım Geliştirme Modelleri

Günümüzde kullanılan birçok model bulunmaktadır. İlk model olan şelale modeli ile günümüzde çok kullanılan çevik modele değinilecektir.

Şelale Modeli

Şelale modeli (waterfall model) 1970 yılında Winston W. Royce tarafından tanımlanmıştır. Bu modelde bir sonraki aşamaya geçmek için, bir önceki aşamanın tamamlanması gerekir. Bir sonraki aşamaya geçmeden, içinde bulunulan aşamanın tüm dokümanları hazırlanır. Bu model, **klasik yaşam döngüsü (classic life cycle)**, **doğru sıralı (linear sequential)** model olarak da adlandırılır.

Şelale modelinin aşamaları;

1. **Gereksinim Analizi (requirement analysis)**: Müşteri ihtiyaçları toplanır ve belgelenir. Projenin kapsamı belirlenir, Projenin hedef ve amaçları belirlenir. Projenin kaynakları planlanır. Fonksiyonel ve teknik gereksinimler belirlenir. Gereksinimler gözden geçirilir ve kabul koşulları belirlenir.
2. **Sistem ve Yazılım Tasarımı (design)**: Yazılım mimarisi ve sistem yapısı planlanır. Veri tabanı, ara yüz, iş mantığı gibi bileşenler tasarlanır. Düşük ve yüksek düzey tasarım yapılır. **Düşük Seviye Tasarım (Low Level Design-LLD)**, yazılım geliştirme sürecinde detaylı sistem bileşenlerinin ve bunların etkileşimlerinin belirtildiği ve bunların uygulandığı bir aşamadır. **Yüksek Seviye Tasarım (High Level**

³⁷ <https://www.geeksforgeeks.org/software-development-life-cycle-sdlc/>

Design-HLD), bir sistemin genel yapısının planlandığı uygulamaların geliştirilmesinde ilk adım olup, esas olarak sistemin farklı bileşenlerinin, kodlama ve uygulama hakkında bilgi sahibi olmadan birlikte nasıl çalıştığına odaklanır. Bu, projede yer alan herkesin hedefleri anlamasına yardımcı olur ve geliştirme sırasında iyi iletişim sağlar.

3. **Uygulama ya da Kodlama (implementation)**: Tasarlanan yapıya uygun olarak yazılım geliştirme yapılır. Kod standardı belirlenir yani programlama dilleri burada belirlenir. Kod ölçeklendirme yapılır. Kod ölçeklendirme, uygulamanın daha fazla kullanıcı, daha fazla veri veya daha yoğun trafik ile etkili şekilde çalışabilmesi için yapılan düzenlemelerdir. Sürüm kontrolü belirlenir. Kodun gözden geçirilmesi için süreç belirlenir.
4. **Test Etme (testing)**: Kodlanan yazılım test edilir. Genel sistem testi yanında manuel test ve otomatik testler yapılabilir. Varsa hatalar düzeltilir, kalite kontrolü sağlanır.
5. **Dağıtım (deployment)**: Yazılımın uygun sürümü son kullanıcıya sunulur. Kurulumlar ve eğitimler yapılabilir.
6. **Bakım (maintenance)**: Kullanım sırasında oluşan hatalar düzeltilir. Geri bildirimler sonrası güncellemeler ve iyileştirmeler yapılır.

Bu modelde yazılıma ilişkin dokümantasyon çok net ve kapsamlıdır. Bir aşama tamamlanmadan sıradakine geçilmez. Süreç kontrollüdür. Gereksinimi değişmeyen büyük projelerde kullanılır. Yönetimi ve izlenebilirliği kolaydır. Ancak geri dönmek zordur. Bir hata sonradan fark edilirse süreç baştan sarar. Değişen gereksinimler varsa uygun değildir. Bu nedenle esnek değildir. Teslimat çok geç ortaya çıkar yani müşteri ürünü en sonunda görür.

Çevik Model

Çevik model (agile model), şelale modelinin aksine temel olarak proje süresince gelen değişen isteklere hızla uyum sağlamak için tasarlanmıştır. Günümüzde çok tercih edilen modellerden biridir. Bu model, yazılım geliştirme süreçlerinde esnekliği, müşteri memnuniyetini ve hızlı teslimatı ön planda tutan **tekrarlamalı (iterative)** bir yaklaşımdır.

Büyük bir projeyi küçük parçalara ayırarak, adım adım geliştirme ve sürekli geri bildirimle iyileştirme sağlar. Bu model, değişen müşteri ihtiyaçlarına hızlıca adapte olabilmek için tasarlanmış, sürekli iletişim, erken teslimat ve kademeli gelişim ilkelerine dayanan bir geliştirme modelidir.

Çevik modelde projeyi yönetmek için proje personelinin bazı **değerleri (values)** benimsemesi gerekir.

1. Bireyler ve etkileşimler, süreçler ve araçlardan daha önemlidir.
2. Çalışan yazılım, kapsamlı dokümantasyondan daha önemlidir.
3. Müşteri ile iş birliği, sözleşme pazarlığından daha önemlidir.
4. Değişime karşılık vermek, bir planı izlemekten daha önemlidir.

Bu değerler ile 12 adet ilkeye uyulur. Bunlar **çevik manifestoda (agile manifesto)** yer alır;

1. Müşteri memnuniyeti, erken ve sürekli yazılım teslimatıyla sağlanır. Müşteriye sık aralıklarla çalışan yazılım sunarak geri bildirim alınır ve güven kazanılır.
2. Değişen gereksinimler, projenin geç aşamalarında bile kabul edilir. Çevik süreçler, değişime yanıt vererek müşterinin rekabet avantajını korumasına yardımcı olur.
3. Çalışan yazılım, birkaç haftada bir, mümkün olan en kısa sürede teslim edilir. Kısa döngülerle (sprintlerle) geliştirme yapılır.
4. İş birliği, iş birimlerinden (müşteriden) ve geliştiricilerden oluşan bir ekip arasında sürekli olmalıdır. Müşteri ve geliştirici ekip sürekli iletişim halinde olur.
5. Motivasyonu yüksek bireyler etrafında projeler inşa edilir. Ekip üyelerine güvenilir, destekleyici bir ortam sunulur ve onlara güvenilir.
6. Yüz yüze iletişim, bilgi aktarımının en etkili yoludur. Özellikle aynı fiziksel ortamda çalışan ekiplerde hızlı karar alınmasını sağlar.
7. Çalışan yazılım, ilerlemenin birincil ölçüsüdür. Raporlar ya da belgeler değil, gerçek çalışan yazılım önemlidir.
8. Çevik süreçler sürdürülebilir geliştirmeyi teşvik eder. Ekip, sabit bir hızda uzun süre çalışabilmelidir. Aşırı mesai değil, dengeli tempo hedeflenir.

9. Teknik mükemmellik ve iyi tasarım, çevikliği artırır. Kaliteli kod ve temiz mimari, değişimlere uyumu kolaylaştırır.
10. Basitlik esastır – yapılmaması gereken işleri yapmaktan kaçınmak. Gereksiz özellik ve aşırı mühendislikten uzak durulur.
11. En iyi mimariler, gereksinimler ve tasarımlar, kendi kendini organize eden ekiplerden çıkar. Ekip içindeki bireyler, en uygun çözümleri birlikte bulur.
12. Ekip düzenli aralıklarla nasıl daha verimli olabileceğini düşünür ve davranışlarını buna göre uyarlar. **Sürekli iyileştirme (retrospective)** ile gelişim sağlanır.

Çevik proje yönetiminde ön tanımlı roller vardır;

1. **Product Owner (Ürün Sahibi)**: Müşteri ihtiyaçlarını belirler. Ürünün ne olacağına karar verir, iş önceliklerini belirler.
2. **Scrum Master**: Takımın çevik kurallarına uymasını sağlar. Engelleri kaldırır, kolaylaştırıcı rolündedir.
3. **Development Team (Geliştirme Takımı)**: Yazılımı analiz eder tasarlar, kodlar, test eder.

Çevik modelde proje, sprint adı verilen kısa geliştirme döngülerine ayrılır. Her sprint sonunda çalışan bir yazılım parçası müşteriye teslim edilir. Bunun için;

- **Product Backlog** yani (**İş Listesi**) hazırlanır.
- Her sprint için **Sprint Planning** toplantısı yapılır.
- Yazılım takımı, 1 ila 4 haftalık sprint süresince belirlenen işleri yapar.
- **Daily Scrum** yani günlük toplantılar ile ilerleme izlenir.
- **Sprint Review** ile müşteriye sunum yapılır.
- **Sprint Retrospective** ile ekip kendi sürecini değerlendirir ve iyileştirir.

Bir kurumda çevik proje yönetimi yapabilmek için kurumun buna hazırlanması gerekir;

- Kurum Kültürü: Çevik proje yönetimi, hiyerarşik ve katı yönetim anlayışı olan kurumlarda zor olur. Bu yüzden üst yönetimin çevik felsefesini benimsemesi gerekir. Takımlara güven, şeffaflık ve sürekli iyileştirme desteklenmelidir. "Komuta-kontrol" yerine "hizmetkar liderlik" yaklaşımı benimsenmelidir.
- Kurum İçi Eğitim: Tüm ilgili ekipler (yazılım geliştiriciler, testçiler, analistler, ürün sahipleri, yöneticiler) çevik eğitimi almalıdır. Scrum, Kanban, Lean gibi çerçeveler tanıtılmalı ve hangi modelin kuruma uygun olduğuna karar verilmelidir.
- Roller: Scrum kullanılıyorsa roller atanmalıdır; Product Owner, Scrum Master, Development Team.
- Takım: Küçük, çapraz fonksiyonel ve **çok disiplinli (multi-disipliner)** takımlar oluşturulmalıdır. Her takım kendi içinde analiz, tasarım, geliştirme ve test yapabilir durumda olmalıdır.
- Sprint Yönetimi: Scrum uygulanıyorsa;
 - o **Sprint Planning (Sprint Planlama)**: İşlerin belirlenmesi ve sprint hedefinin tanımlanması.
 - o **Daily Stand-up (Günlük Toplantı)**: Kısa günlük toplantılarla durum değerlendirmesi.
 - o **Sprint Review**: Ürün sahipleri ve paydaşlarla yapılan toplantı (**demo**).
 - o **Sprint Retrospective**: Ekip içi iyileştirme toplantısı.
- Agile süreçleri destekleyen yazılımlar: Jira, Trello, Azure DevOps, ClickUp, Asana gibi araçlarla iş takibi yapılabilir. CI/CD altyapıları kurulmalıdır.
- Pilot Proje Deneyimi: Öncelikle küçük bir projede çevik yöntem uygulanır. Deneyim kazanılır, süreçler test edilir. Başarılar ve hatalar analiz edilerek diğer ekiplerde yaygınlaştırılır.
- Sürekli İyileştirme (**Kaizen**): Ekipler düzenli olarak süreçlerini gözden geçirip iyileştirme önerileri üretmelidir. Geri bildirim döngüleri (müşteriden ve ekip içinden) önemlidir.
- Başarı ve Performans Ölçütleri: Çevik ekiplerin performansı birey bazında değil, takım bazında ölçülür. Teslim süreleri, müşteri memnuniyeti, iş değeri odaklı metrikler kullanılır.

Çevik modelde müşteri memnuniyeti yüksektir. Hızlı teslimat ve erken geri bildirim alınır. Değişime açık ve uyumlu yazılım geliştirilir. Takım içi iletişim güçlüdür. Kalite artar, çünkü sürekli test edilir. Ancak sürekli müşteri katılımı gerekir. Belirsiz projelerde zaman ve maliyet tahmini zordur. Deneyimsiz ekiplerde karmaşıklık ortaya çıkar. Dokümantasyon önemsiz gibi görülüp ihmal edilebilir.

Modern Yazılımları Özellikleri

Günümüzde geliştirilen modern uygulamaların üç temel özelliği vardır:

1. Katmanlı mimari (**layered architecture**): İstemci/sunucu ya da n katmanlı model. Katman sayısı arttıkça **karmaşıklık** (**complexity**) azalır.
2. Bileşen kullanımı (**component using**): Bileşen kullanımı yazılımda **tekrar kullanılabilirliği** (**reusing**) artırır.
3. Dağıtık programlama (**distributed programming**): **Ölçeklenebilirlik** (**scalability**) artar. Yazılım birden fazla bilgisayar üzerine çeşitli görevleri dağıtarak çalışır.

Modern kurumsal yazılımlarda, kurumun gerçek dünyada yaptığı işi, elektronik ortamda **iş katmanı** (**business layer**) yapar. Buradan hareketle kurumsal bir yazılıma başlamadan önce, süreçlerin incelenerek **detaylı modellenmesi** (**business model**) gerektiği ortaya çıkar. İş modelinin ortaya çıkarılmasına yönelik yapılan çalışmalara **iş analizi** (**business analysis**) adı verilir. Burada dikkat edilmesi gereken, iş analizinin doğru yapılmasıdır. İş analizinin yanlış yapılması, yanlış bir yazılımın ortaya çıkacağı anlamına gelir.

Katmanlı Mimari

Katman kavramı iki türlü düşünülmelidir. Kodların gruplanarak birbirinden ayrıldığı katmanlara **kod katmanı** (**layer**) adı verilir. Fiziksel olarak birbirinden ayrılmış kodlara ise **fiziksel katman** (**tier**) adı verilir. Kısaca derlenmiş olarak birbirinden ayrı olan katmanlar fiziksel katman, diğer tip katman ise kod katmanı olarak adlandırılır. Katman sayısı göz önüne alınarak uygulamalar aşağıdaki şekilde sınıflandırılabilir.

Yekpare Uygulamalar

Genel olarak hiç katmanı olmayan, tek bir çalıştırılabilir dosyadan oluşan uygulamalar **yekpare** (**monolithic**) uygulamalar olarak adlandırılır. Kullanıcı tanımlı sınıf kütüphanesi olmayan tüm konsol ve Windows uygulamaları bu sınıfa girer. Şu ana kadar verilen örneklerin çoğu bu kapsamda değerlendirilebilir. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

KonsolUygulaması	→ Çözüm (Solution)
└─ Yekpare	→ Proje (Console Application)
└─ Program.cs	→ Ana (Main) Program

Projeyi oluşturmak için;

1. Visual Studio üzerinde *File→New→Project...* seçilerek **KonsolUygulaması** adlı çözüm içinde **Yekpare** adlı projemiz *Console Application* olarak oluşturulur.
2. Oluşan projede **Program.cs** dosyası otomatik olarak oluşur. Bu dosyayı aşağıdaki şekilde değiştiriyoruz;

```
namespace Yekpare
{
    class Server
    {
        public void Sunucuİşlevi()
        {
            Console.WriteLine("Sunucu İşlevi Çağrıldı.");
        }
    }
    class Client
    {
        static void Main(string[] args)
        {
            Server sunucu = new Server();
            sunucu.Sunucuİşlevi();
        }
    }
}
```

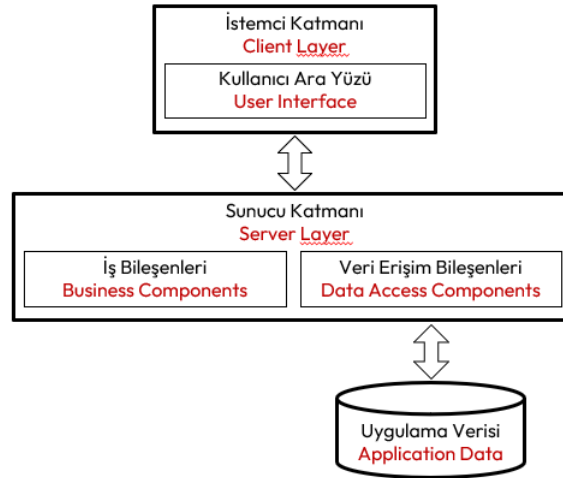
Yekpare uygulamalar, her ne kadar veri tabanı kullansalar da bu tür yazılımlar tek bir proje olarak kodlanır ve derlenir. Proje genellikle az sayıda kod dosyasından oluşur. Seksenli yıllara kadar geliştirilen uygulamalar genellikle bu yapıdadır.

Bilgisayara yaptırılan işlerin kapsamının zamanla daha fazla genişlemesi, kurumların iş süreçlerini daha çok içermesi ve uygulamanın işletme ve bakım maliyetlerinin artmasıyla birlikte, bu yazılımları uygun katmanlara bölünmesi ihtiyacı doğurmuştur.

İki Katmanlı İstemci-Sunucu Uygulamaları

İstemci-Sunucu (client-server) mimarisinde uygulama iki katmanlı olup, genellikle veri katmanı bir bilgisayarda diğer katman olan istemci katmanı ise kullanıcı bilgisayarlarında bulunur.

Bu tür uygulamalarda kullanıcı ara yüzü bileşenleri (User Interface Components-UIC) istemci (client) katmanını oluşturur. Geri kalan iş süreçleriyle ilgili geliştirilen bileşenler ile veri tabanı ile ilgili iletişimi sağlayacak olan bileşenler ise sunucu (server) katmanını oluşturur.



Şekil 62. İstemci-Sunucu Mimarisi

Örnek olarak oluşacak uygulamanın yapısı aşağıda verilmiştir;

İstemciSunucuUygulaması	→ Çözüm (Solution)
└ İstemciBileşenleri	→ Proje (Console Application)
└ Program.cs	→ Ana (Main) Program
└ SunucuBileşenleri	→ Proje (Class Library)
└ İstemci.cs	→ Class

Projeyi oluşturmak için;

1. Visual Studio üzerinde *File→New→Project...* seçilerek **İstemciSunucuUygulaması** adlı çözüm içinde **İstemciBileşenleri** adlı projemiz *Console Application* olarak oluşturulur.
2. Oluşan projede **Program.cs** dosyası otomatik olarak oluşur.

```
namespace İstemciBileşenleri
{
    class Client
    {
        static void Main(string[] args)
        {
        }
    }
}
```

3. Daha sonra sunucu katmanı yapmak için *Solution Explorer* üzerinde **KonsolUygulaması** çözümüne sağ tıklanarak *Add→New→Project...* seçerek **SunucuBileşenleri** adlı *Class Library* oluşturulur
4. Proje ile oluşan **Class1.cs** dosyasının adı **Sunucu.cs** olarak değiştirilir ve dosyada aşağıdaki değişiklikler yapılır;

```
namespace SunucuBileşenleri
{
```



```

public class Server {
    public void Sunucuİşlevi(){
        Console.WriteLine("Sunucu İşlevi Çağrıldı.");
    }
}

```

- İstemci tarafında sunucu bileşenlerini kullanmak için *Solution Explorer* üzerinde **SunucuBileşenleri** projesi sağ tıklanarak *Build* seçilir ve derlenir.
- Ardından **İstemciBileşenleri** projesi seçilir ve *Add→Project Reference→Project...* ile sunucu projemiz istemci projemize dahil edilir.
- Bunu yaptıktan sonra **İstemciBileşenleri** projesinde aşağıdaki değişiklikler yapılır;

```

using Sunucu; //Bunu yapabilmek için:İstemciye Sunucu Referans olarak eklenir.
namespace İstemciBileşenleri
{
    class Client {
        static void Main(string[] args){
            Sunucu.Server sunucu = new Sunucu.Server();
            sunucu.Sunucuİşlevi();
        }
    }
}

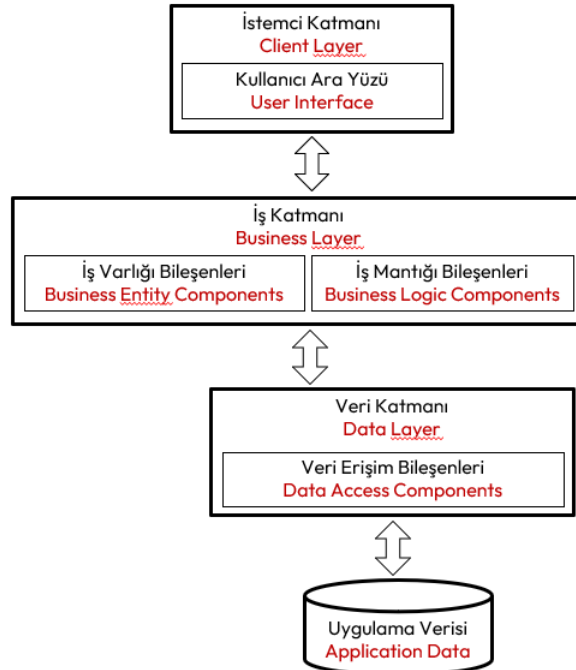
```

Artık programımız çalışır.

Üç Katmanlı Uygulamalar

Nesne yönelimli yazılım geliştirme ile bildikte nesneler arasındaki iletişimin bir başka katman olarak tasarlandığı uygulamalardır. İki katmanlı mimariden farklı olarak kullanıcı ara yüzü ile iş bileşenleri birbirinden ayrılarak, kullanıcılara istemci yükleme işleminin çok kolaylaştığı uygulamalardır.

İstemci-sunucu yazılımlarında zamanla, sunucu tarafında **iş süreçlerini modelleyen bileşenler** (**Business Entity Components-BEC**) ile veri tabanı ile iletişimi sağlayan bileşenler (**Data Access Components-DAC**) birbirinden ayrılarak farklı iki katman olarak geliştirilmiştir.



Şekil 63. Üç Katmanlı Mimari

Örnek olarak oluşacak uygulamanın yapısı aşağıda verilmiştir;

ÜçKatmanlıUygulama
└─ İstemciBileşenleri

→ Çözüm (Solution)
→ Proje (Console Application)

└─ Program.cs	→ Ana (Main) Program
└─ İşBileşenleri	→ Proje (Class Library)
└─ İşSınıfı.cs	→ Class
└─ VeriBileşenleri	→ Proje (Class Library)
└─ Veritabanı.cs	→ Class

Projeyi oluşturmak için;

1. Visual Studio üzerinde *File→New→Project...* seçilerek **ÜçKatmanlıUygulama** adlı çözüm içinde **İstemciBileşenleri** adlı projemiz *Console Application* olarak oluşturulur.
2. Oluşan projede **Program.cs** dosyası otomatik olarak oluşur.

```
namespace İstemciBileşenleri
{
    class Client
    {
        static void Main(string[] args)
        {
        }
    }
}
```

3. Daha sonra sunucu katmanı yapmak için *Solution Explorer* üzerinde çözüm adına sağ tıklanarak *Add→New→Project...* seçerek **İşBileşenleri** adlı *Class Library* oluşturulur.
4. Proje ile oluşan **Class1.cs** dosyasının adı **İşSınıfı.cs** olarak değiştirilir ve dosyada aşağıdaki değişiklikler yapılır;

```
namespace İşBileşenleri
{
    public class İşSınıfı
    {
        public void işYöntemi()
        {
            // İş mantığı burada yer alır
        }
    }
}
```

5. *Solution Explorer* üzerinde çözüm adına sağ tıklanarak *Add→New→Project...* seçilir ve **VeriBileşenleri** adında *Class Library* oluşturulur.
6. Proje ile oluşan **Class1.cs** dosyasının adı **Veritabanı.cs** olarak değiştirilir ve dosyada aşağıdaki değişiklikler yapılır;

```
namespace VeriBileşenleri
{
    public class Veritabanı
    {
        public void veriEkle(int veri)
        {
            // Veri ekleme işlemleri burada yer alır
            Console.WriteLine($"Veri tabanına veri ekleme {veri} işlemi çalıştı.");
        }
        public void veriSil(int veri)
        {
            // Veri silme işlemleri burada yer alır
            Console.WriteLine("Veri tabanından veri silme {veri} işlemi çalıştı.");
        }
    }
}
```

7. *Solution Explorer* üzerinde **VeriBileşenleri** projesi sağ tıklanarak *Build* seçilir ve derlenir.

8. Daha sonra *Solution Explorer* üzerinde **İşBileşenleri** projesi seçilir ve *Add→Project Reference→Project...* seçilerek **VeriBileşenleri** projesi seçilir. Bunu yaptıktan sonra **İşBileşenleri** tarafında aşağıdaki değişiklikleri yapabiliriz.

```
using VeriBileşenleri; // İş projesine Veri Projesi Referans olarak eklenir.
namespace İşBileşenleri
{
    public class İşSınıfı
    {
        public void işYöntemi()
        {
            // İş mantığı burada yer alır
            int işlenenVeri = 0;
            VeriTabanı veriTabanı = new VeriTabanı(); //
            veriTabanı.veriEkle(işlenenVeri); //
            //...
            veriTabanı.veriSil(işlenenVeri); //
            Console.WriteLine("İş sınıfı iş yöntemi çalıştı.");
        }
    }
}
```

9. *Solution Explorer* üzerinde **İşBileşenleri** projesi sağ tıklanarak *Build* seçilir ve derlenir.
10. Daha sonra *Solution Explorer* üzerinde **İstemciBileşenleri** projesi seçilir ve *Add→Project Reference→Project...* seçilerek **İşBileşenleri** projesi seçilir. Bunu yaptıktan sonra **İstemciBileşenleri** tarafında aşağıdaki değişiklikleri yapabiliriz.
11. Artık projemiz çalışabilir durumdadır.

```
using İşBileşenleri; // İstemci projesine İş Projesi Referans olarak eklenir.
namespace İstemciBileşenleri
{
    class Client
    {
        static void Main(string[] args)
        {
            İşSınıfı işnesnesi = new İşSınıfı(); //
            işnesnesi.işYöntemi(); //
        }
    }
}
```

Çok Katmanlı Uygulamalar

Yukarıda örneği verilen üç katmanlı uygulamalarda ihtiyaca göre katmanlar daha alt katmanlara ayrılabilir.

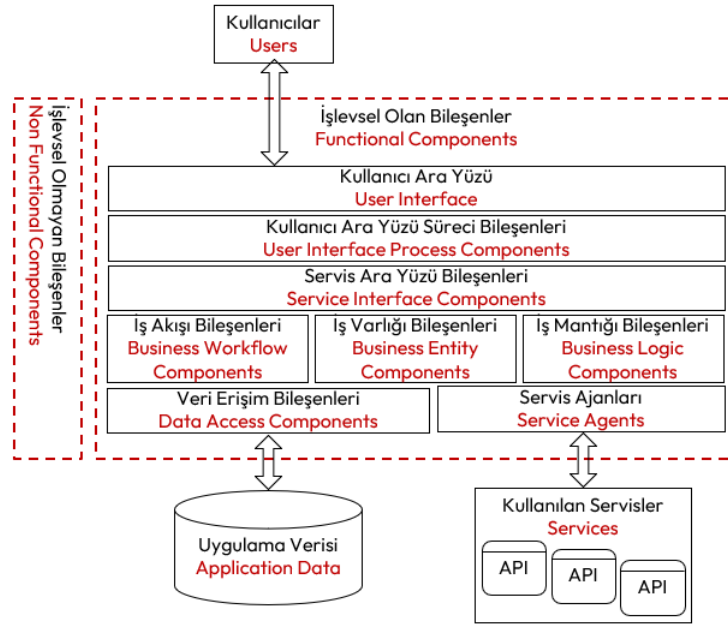
Örneğin kullanıcı ara yüzü katmanı, aşağıdaki bileşenlerden oluşacak şekilde ikiye ayrılabilir;

1. İnsanların yazılıma talimatlarını verdiği **insan talimatları bileşenleri** (**Human Instruction Components-HIC**) ve
2. Kullanıcı talimatlarını, kullanıcı ara yüzünden alıp, iş katmanının anlayacağı şekle çeviren **kullanıcı süreci bileşenleri** (**User Process Components-UPC**).

Benzer şekilde İş katmanı, aşağıdaki bileşenlerden oluşacak şekilde üçe ayrılabilir;

1. **İş akışını sağlayan bileşenler** (**Business Workflow Components-BWC**),
2. **İş varlıkları bileşenleri** (**Business Entity Components-BEC**) ve
3. İş varlıklarının toplu olarak işlenmesi ile ilgili **iş mantığı bileşenleri** (**Business Logic Components-BLC**)
4. Çok katmanlı mimari, günümüz yazılım mimarisi olup mümkün olduğunca katmanların birbirinden bağımsız tasarlandığı uygulamalardır. Üç katmanlı mimariden daha alt katmanlara ayrılarak geliştirilen yazılımlara, n-katmanlı (n-layer/ n-tier) yazılım adı verilir.

Çok katmanlı yazılım geliştirilirken kullanılan bileşenler ve bu bileşenlerin bulunduğu katmanlar detaylı olarak aşağıdaki şekilde verilmiştir.



Şekil 64. Çok Katmanlı Mimari

Yukarıda verilen şekilde, dikine yazılı olan yerler, yazılımı gerçekleştiren sistemde bulunmayan, ancak yazılım geliştirilirken ihtiyaç duyulan bileşenleri ve bununla ilgili servisleri temsil etmektedir. Bu nedenle bu bileşenler, yazılımın **işlevsel olmayan** (non-functional) bölgeleridir. Yani ortada bir yazılım olmayınca hiçbir önemi olmayan bileşenlerdir. Geri kalan bileşenler ise **işlevsel** (functional) bileşenler olarak adlandırılır. İşlevsel olmayan bileşenler detaylı olarak aşağıda gösterilmiştir.



Şekil 65. İşlevsel Olmayan Bileşenler

Burada bahsedilen **güvenlik** (security) katmanlara **rol bazlı** (role based) veya **kod erişimi** (code access) olmak üzere iki şekilde ilave edilir.

Sonuç olarak yazılımlarda katman sayısı arttıkça **performans** (performance) düşer ama **esneklik** (flexibility) artar, **bakım** (maintainability) kolaylaşır, **yeniden kullanılabilirlik** (reusability) ve **ölçeklenebilirlik** (scalability) sağlanmış olur.

Katmanların Tasarlanması

İş modeli (business model), kurumun yaptığı işi anlatan bir modeldir. Bu modelin çıkarılmasına yönelik yapılan analize ise **iş analizi** (business analysis) adı verilir. İş modelinde yer alan bütün süreçler doğal olarak bilgisayar tarafından yapılamaz. İşte iş modelindeki bilgisayar tarafından süreçleri içeren analize ise **gereksinim analizi** (requirement analysis) adı verilir.

Yapılan gereksinim analizinden sonra katmanlar ayrı ayrı tasarlanır;

1. **Business Logic Components (BLC)**, **sorun alanı bileşenleri (Problem Domain Components-PDC)** olarak da adlandırılır. Uygulamanın, iş modelinde belirtilen, iş kurallarını içeren bileşenlerdir. PDC, gerçek dünyadaki problemin yazılımda modellenmiş haliyle ilgilidir. Sistem çözümünün temel mantığını ve **iş kurallarını (business rules)** kapsar. Gerçek hayat problemini, teknik detaylardan bağımsız şekilde doğru ve tam olarak modellemek amacıyla tanımlanırlar.

```
//Örnek PDC/BLC Bileşenleri
public class Sipariş
{
    public Guid Id { get; private set; }
    public List<SiparişKalemi> SiparişKalemleri { get; private set; }
    public DateTime SiparişZamanı { get; private set; }
    public Sipariş()
    {
        Id = Guid.NewGuid();
        SiparişKalemleri = new List<SiparişKalemi>();
        SiparişZamanı = DateTime.UtcNow;
    }
    public void YeniKalemEkle(SiparişKalemi item)
    {
        if (item.Miktar <= 0)
            throw new ArgumentException("Miktar 0 dan fazla olmalıdır!");

        SiparişKalemleri.Add(item);
    }
    public decimal ToplamSiparişÜcretiGetir()
    {
        return SiparişKalemleri.Sum(i => i.BirimFiyat * i.Miktar);
    }
}

public class SiparişKalemi
{
    public Guid KalemId { get; set; }
    public int Miktar { get; set; }
    public decimal BirimFiyat { get; set; }
}
```

2. **Business Entity Components (BEC)** veri tabanı tabloları ile eşleşen ve iş nesnelerini temsil eden sınıflar içerir. Sınıflar yalnızca özellik barındırır ve veri taşıma, saklama veya iletme süreçlerinde kullanılır, içinde iş mantığı barındırmaz. Bu sınıflar çok nadir olarak **doğrulama (validation)** yöntemleri içerir. Bu sınıflardan imal edilen nesneler **veri transfer nesneleri (Data Transfer Object-DTO)** olarak kullanılır.

```
//Örnek BEC Bileşenleri
public class SiparişDto
{
    public Guid Id { get; set; }
    public List<SiparişKalemiDto> SiparişKalemleri { get; set; }
    public decimal ToplamÜcret { get; set; }
    public DateTime SiparişZamanı { get; set; }
}

public class SiparişKalemiDto
{
    public Guid KalemId { get; set; }
    public int Miktar { get; set; }
    public decimal BirimFiyat { get; set; }
}
```

3. PDC bileşenleri, iş süreçlerini işlerken kullanılabilir; fakat veri tabanına yazılacağı veya servis üzerinden bir istemciye gönderileceği zaman iş varlığına (BEC) dönüştürülür. Dönüştürme için ayrı bileşenler tasarlanır.

```
//PDC den BEC bileşenine dönüştürme
public static class SiparişMapper
{
    public static SiparişDto ToDto(Sipariş sipariş)
    {
        return new SiparişDto
        {
            Id = sipariş.Id,
            SiparişZamanı = sipariş.SiparişZamanı,
            SiparişKalemleri = sipariş.SiparişKalemleri.Select(i => new SiparişKalemIdto
            {
                KalemId = i.KalemId,
                Miktar = i.Miktar,
                BirimFiyat = i.BirimFiyat
            }).ToList(),
            ToplamÜcret = sipariş.ToplamSiparişÜcretiGetir()
        };
    }
}
```

4. **Business Workflow Components (BWC)**, kurumsal yazılım mimarilerinde **iş süreçlerinin** (**business processes/workflows**) akışını ve yönetimini tanımlayan bileşenlerdir. Genellikle iş akışlarının adım adım yürütülmesi, durumların yönetilmesi ve süreç bazlı kararların verilmesi gibi konuları kapsar.

```
public class SiparişWorkflow
{
    private readonly IÖdemeServisi ödemeServisi;
    private readonly IStokServisi stokServisi;
    private readonly IGönderimServisi gönderimServisi;

    public OrderWorkflow(IÖdemeServisi ödeme, IStokServisi stok, IGönderimServisi gönderim)
    {
        ödemeServisi = ödeme;
        stokServisi = stok;
        gönderimServisi = gönderim;
    }

    public async Task ProcessOrderAsync(Sipariş sipariş)
    {
        if (!await ödemeServisi.Charge(sipariş))
            throw new Exception("Ödeme Alınamadı!");

        if (!await stokServisi.CheckStock(sipariş.SiparişKalemleri))
            throw new Exception("Yetersiz Stok!");

        await gönderimServisi.DağıtımHazırla(sipariş);

        sipariş.Durum = SiparişDurumu.DağıtımaÇıkacak;
    }
}
```

5. **Service Interface Components (SIC)**, uygulamanın dış dünya (kullanıcılar, sistemler, API istemcileri vs.) ile iletişim kurduğu giriş noktalarıdır. Diğer bir deyişle, uygulamanın hizmetlerini dışa açan ara yüzlerdir. RESTful API Başlığını inceleyiniz.
6. **Data Access Components (DAC)**, iş varlıklarının (BEC) tuttuğu verilere ilişkin bir kalıcılık sağlamak için veritabanı ya da diğer kalıcı veri kaynaklarına erişimi soyutlayan ve yöneten bileşenlerdir. Kalıcılığı sağlamak için bu bileşenler bazen detaylı bir mantık içerebilir. Bu mantığa, **kalıcılık mantığı** (**persistence logic**) adı verilir. Bunlar sayesinde uygulamanın diğer katmanları (özellikle uygulama ve iş mantığı katmanları), veri kaynağının teknik detaylarını bilmeden veri okuma/yazma işlemlerini gerçekleştirebilir. Burada veri işlemek için **depo deseni** (**repository pattern**) kullanılır. Bu desende CRUD işlevlerini de içine alan **ara yüzler** (**interface**) üzerinden iş bileşenleri (BEC) ihtiyaçları çözülür.

```
public interface ISiparişRepository
```

```

{
    Task<Sipariş?> GetByIdAsync(Guid id);
    Task AddAsync(Sipariş sipariş);
    Task SaveChangesAsync();
    Task<List<Sipariş>> GetAllAsync();
    Task DeleteAsync(Guid id);
}

public class SiparişRepository : ISiparişRepository
{
    private readonly string _connectionString;
    public SiparişRepository(string connectionString)
    {
        _connectionString = connectionString;
    }
    public async Task<List<Sipariş>> GetAllAsync()
    {
        var sipariş = new List<Sipariş>();
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand(
            "SELECT Id, SiparişZamanı FROM Siparişler",
            conn);
        await conn.OpenAsync();
        using var reader = await cmd.ExecuteReaderAsync();
        while (await reader.ReadAsync())
        {
            sipariş.Add(new Sipariş
            {
                Id = reader.GetGuid(0),
                SiparişZamanı = reader.GetDateTime(2)
            });
        }
        return sipariş;
    }
    public async Task<Sipariş?> GetByIdAsync(Guid id)
    {
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand(
            "SELECT Id, SiparişZamanı FROM Siparişler WHERE Id = @Id",
            conn);
        cmd.Parameters.AddWithValue("@Id", id);
        await conn.OpenAsync();
        using var reader = await cmd.ExecuteReaderAsync();
        if (await reader.ReadAsync())
        {
            return new Sipariş
            {
                Id = reader.GetGuid(0),
                SiparişZamanı = reader.GetDateTime(2)
            };
        }
        return null;
    }
    public async Task AddAsync(Sipariş order)
    {
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand(
            "INSERT INTO Siparişler (Id,SiparişZamanı) VALUES (@Id, @SiparişZamanı)",
            conn);
        cmd.Parameters.AddWithValue("@Id", order.Id);
        cmd.Parameters.AddWithValue("@SiparişZamanı", order.SiparişZamanı);
        await conn.OpenAsync();
    }
}

```

```

        await cmd.ExecuteNonQueryAsync();
    }
    public async Task UpdateAsync(Sipariş order)
    {
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand(
            "UPDATE Siparişler SET SiparişDate = @SiparişZamanı WHERE Id = @Id",
            conn);
        cmd.Parameters.AddWithValue("@Id", order.Id);
        cmd.Parameters.AddWithValue("@CustomerName", order.CustomerName);
        cmd.Parameters.AddWithValue("@SiparişDate", order.SiparişDate);
        await conn.OpenAsync();
        await cmd.ExecuteNonQueryAsync();
    }
    public async Task DeleteAsync(Guid id)
    {
        using var conn = new SqlConnection(_connectionString);
        using var cmd = new SqlCommand(
            "DELETE FROM Siparişler WHERE Id = @Id",
            conn);
        cmd.Parameters.AddWithValue("@Id", id);
        await conn.OpenAsync();
        await cmd.ExecuteNonQueryAsync();
    }
}

```

7. Veri erişim bileşenleri (DAC), her BEC ile ilgili veri kaynağında, **yeni kayıt** (**create**), mevcut kaydı **okuma** (**read**), kaydı **güncelleme** (**update**) ve bir kaydı **silme** (**delete**) yöntemlerini yerine getirir. Açıklanan bu işlevleri yerine getiren yöntemlerin tümü kısaca CRUD (**Create-Read-Update-Delete**) olarak adlandırılır. CRUD yöntemleri; ABCD (**Add-Browse-Change-Delete**), ACID (**Add-Change-Inquire-Delete**), BREAD (**Browse-Read-Edit-Add-Delete**) ve VADE (**View-Add-Delete-Edit**) olarak da adlandırılır.

Katmanlar Arası İletişim

Katmanlı mimaride, iş varlığı bileşenleri (BEC) orta katmanda yer aldığından hem kullanıcı katmanında hem de veri erişim katmanında kullanılırlar. Bu nedenle bu katmanlarda iş varlıklarını, verileri, katmanlar arasında taşırlar. Bu taşınma işlemi, genel olarak üç değişik teknoloji ile sağlanır;

- **Sınıf** (**class**): Katmanların birbirine çok yakın olduğu durumda tercih edilir.
- **Veri Seti** (**DataSet**): Katmanlar arası veri iletişimi veri tabanı üzerinden yapılır. Genellikle aynı ağı kullanan katmanlar için tercih edilir.
- **Serileştirme** (**serializing**): Bir nesnenin **durumunu** (**state**) kalıcı veya taşınabilir bir şekle dönüştürme sürecidir. Katmanların birbirinden uzak olduğu durumda tercih edilir.

Buradaki tercihler uygulamanın hızını etkiler. Dolayısıyla iş varlıkları tasarlanırken yukarıdaki maddelerden doğru olanını tercih etmek gerekir. Serileştirme, .Net Framework'te, üç farklı teknoloji ile yapılır;³⁸

1. JSON serileştirmesi, .NET nesnelerini JavaScript Nesne Gösterimi'ne (**JavaScript Object Notation-JSON**) eşler. JSON, web genelinde veri paylaşmak için yaygın olarak kullanılan açık bir standarttır. JSON serileştiricisi varsayılan olarak genel özellikleri serileştirir ve özel ve dahili üyeleri de serileştirmek üzere yapılandırılabilir.
2. XML ve SOAP serileştirmesi yalnızca genel özellikleri ve alanları serileştirir ve veri tipine olan sadakati korumaz. Bu durum, verileri kullanan uygulamayı kısıtlamadan veri göndermek veya tüketmek istediğinizde yararlıdır. XML açık bir standart olduğundan, Web genelinde veri paylaşmak için çekici bir seçimdir. SOAP da açık bir standarttır ve bu da onu çekici bir seçim haline getirir.

³⁸ <https://learn.microsoft.com/en-us/dotnet/standard/serialization/>

3. **İkili serileştirme** (**binary serialization**), veri tipi sadakatini korur, yani nesnenin tam durumu kaydedilir ve **serileştirmeden geri aldığınızda** (**deserialization**) tam bir kopyası oluşturulur. Bu tür serileştirme, bir uygulamanın farklı çağrılar arasında bir nesnenin durumunu korumak için yararlıdır. Örneğin, bir nesneyi katmanlar ortasındaki bir panoya serileştirerek farklı uygulamalar arasında paylaşabilirsiniz. Bir nesneyi bir akışa, bir diske, belleğe, ağ üzerinden vb. serileştirebilirsiniz. Uzaktan erişim, nesneleri bir bilgisayardan veya uygulama alanından diğerine "değere göre" geçirmek için bu serileştirmeyi kullanır. Ancak her uygulama ikili serileştirmeyi aynı şekilde yapmalıdır. Aksi durumda tehlikeli durumlar ortaya çıkar.

Bunların yanında çok katmanlı mimaride, katmanların birinde olan istisna, kullanıcı katmanına kadar **yeniden fırlatılır** (**re-throw**). Aksi takdirde istisna kullanıcı katmanına ulaşmadan program kilitlenir. Kullanıcı istisnanın ne olduğu konusunda bilgi sahibi olamaz. Aşağıda istemci sunucu mimarisinde istisna iletimi örneği verilmiştir.

```
//Sunucu.cs (Class Library)
namespace Sunucu
{
    public class Server {
        public int Bölme(int p1,int p2){
            int ret = 0;
            try {
                ret = p1 / p2;
            } catch (Exception e){
                throw e;
                //hata iletme(rethrow):hata kullanıcı
                //katmanına iletilir.
            }
            return ret;
        }
    }
}
```

Bu sunucu kodunu kullanan istemci uygulaması da aşağıda verilmiştir;

```
//İstemci.cs (Console Application)
using Sunucu; //Bunu yapabilmek için:İstemciye Sunucu Referans olarak eklenir.
namespace İstemci
{
    class Client {
        static void Main(string[] args){
            Sunucu.Server sunucu = new Sunucu.Server();
            sunucu.Bölme(12,0); //Hata burada oluşur.
        }
    }
}
```

Bileşen Kullanımı

Uygulamamızda bir **bileşen** (**component**) kullanacak isek bileşenin çalıştığı altyapıdan bağımsız ve soyut olarak kullanılması gerekir. Burada soyut kullanım, bileşenin ve bileşenin işletim sistemindeki yerinin kesin olarak belirtilmemesi ve her kullanımda derlenmemesi ile sağlanır.

Bütün bunlara göre bileşen kullanımında karşılaşılan beş ana zorluk aşağıda verilmiştir.

1. Platform: **Birlikte çalışabilirlik** (**interoperability**), bileşen kullanımında bir problemdir. Bileşenin çalıştığı işletim sistemi ve geliştirildiği platform, kullanımı etkilemektedir. Bileşeni geliştirildiği platformdan bağımsız kullanabilmeliyiz.
2. **Programlama Dili** (**language**): Her programlama dilinin yetenekleri birbirinden farklıdır. Bileşenin geliştirildiği programlama dili farklılıkları bileşen kullanımında sorun yaratabilir.
3. **Süreç** (**process**): Burada sözü geçen süreç, uygulamanın içinde çalıştığı sanal bir zarftır. Bu zarfın sınırları, işletim sistemi tarafından belirlenir. Bu sürecin, yazılımın modellediği süreçle ilgisi yoktur. **İstemci** (**client**) ile **sunucu** (**server**) **bileşenlerinin** (**component**) içinde çalıştığı süreçler arası

iletişimden (**inter-process communication**) kaynaklanan problemler yaşanabilir. Sunucu bileşenlerinin bulunduğu konuma göre dört değişik süreç iletişimi vardır;

- İç süreç (in-process)**: Sunucu ile istemci bileşenlerinin aynı süreç içinde çalışması durumudur.
 - Dış süreç (out-process)**: Sunucu bileşenleri ile istemci bileşenlerinin aynı bilgisayarda farklı süreçler içinde çalışması durumudur.
 - Ağlar arası süreç (inter-network process)**: Sunucu bileşenleri ile istemci bileşenlerinin aynı ağda olan farklı bilgisayardaki süreçler içinde çalışması durumudur.
 - Uzak süreç (remote process)**: Sunucu bileşenleri ile istemci bileşenlerinin farklı ağda olan farklı bilgisayardaki süreçler içinde çalışması durumudur.
- Konumlama (location)**: Bileşeni, konumunun nerede olduğunu bilmeden kullanamayız. Yani bileşeni kullanırken bileşenin bulunduğu yeri "C:\DLLs\MyDll.dll" gibi doğrudan konum belirten kodlamaları (**hard-coding**) yapmamalıyız. Bileşenin dosya sistemindeki yeri değiştiğinde, bileşene bağlı olarak uygulamamızı yeniden derlememiz gerekebilir.
 - Uyarlama (version)**: Bileşenin bir uyarlaması bir uygulamada farklı, diğer uyarlaması bir başka uyarlamada farklı davranabilir. Bu nedenle uyarlama da bileşen kullanımında bir sorun olarak ele alınır.

Burada bileşen kullanımında bahsedilen zorlukları gideren ve bileşenlerin soyut kullanımını sağlayan teknolojiler vardır. Yani bileşenler, bir mağazadan alıp kullanır gibi **soyut (abstract)** olarak kullanmamızı sağlarlar. Bu teknolojiler aşağıda sıralanmıştır:

- DLL (**Dynamic Linked Library**) teknolojisi,
- COM (**Component Object Model**) teknolojisi,
- COM teknolojisinin gelişmiş hali COM+ teknolojisi,
- DCOM (**Distributed COM**) teknolojisi,
- ActiveX teknolojisi,
- CORBA (**Common Object Request Broker Architecture**) veya RMI (**Remote Method Invocation**) teknolojisi.

Microsoft, bileşen kullanımındaki sıkıntıları kronolojik olarak geliştirerek çözmüştür. DLL teknolojisi ile konumlama sorununu çözmüştür. Ardından COM teknolojisi ile programlama dili, süreç, konumlama ve uyarlama problemlerini çözmüştür. .NET teknolojisi ile platform dâhil tüm problemleri çözmüştür.

Dağıtık Programlama

Katmanların birbirinden uzak bulunduğu yazılımlara **dağıtık uygulama (distributed application)** adı verilir. Dağıtık yazılımlarda, bütün işler tek bir kod, hatta tek bir bilgisayar tarafından yürütülmez. Her bir katman ayrı bir bilgisayar ya da süreç tarafından koşturulur. İstemci ile sunucunun birbirinden ayrıldığı dağıtık yazılımlara birçok sebepten dolayı ihtiyaç duyarız. Bunlardan birkaçı aşağıda sıralanmıştır;

- Bazı karmaşık programlar dağıtık bilgisayarlar üzerinde çalışır.
- Birçok ucuz bilgisayar üzerinde bulunan işlemcileri kullanarak çoklu işlemci ile çalışmanın birçok üstünlüğü vardır.
- Dağıtık programlama aynı zamanda **ölçeklenebilirliği (scalability)** de artırır.
- Bazı yazılımlar vardır ki sadece belli donanımlara sahip bilgisayarlarda çalışırlar.
- Güvenlik gerekçesi ile bazı servisler ayrı bilgisayarlarda olmak zorundadır.
- Servislerin sadece kaynak olarak kullanıldığı durumlar.

Dağıtık Programlama Teknolojileri

Dağıtık uygulamalarda, sunucu bileşenlerinin istemci bileşenleri ile iletişim sağlayabilmesi için bizi günümüze kadar getiren birçok teknoloji bulunmaktadır;

- **İç süreç (in-process)** için bir gereklilik bulunmaz.
- **Dış süreç (out-process)** için iletişim, süreçler arası iletişim olan **Inter-Process Communication (IPC)** ile sağlanır. Clipboard, Message Queue and Send Message, Shared Memory, **Lightweight Procedure Call (LRPC)**, **Remote Procedure Call (RPC)**, COM, DCOM, CORBA ve RMI teknolojileri bunu sağlar.

- Ağlar arası süreç (inter-network process) için Jakarta XML Binding (JAXB) ve .NET Remoting teknolojileri kullanılmaktadır.
- Uzak süreç (remote process) için iletişimi için Windows Communication Foundation (WCF) ve Java Architecture for XML Web Service (JAX-WS)

Günümüzde bağlantı hızlarının artması ve bilgisayarların işlem kapasitesinin artması nedeniyle eskimiş kabul edilmektedir. Yeni teknolojiler aşağıda sıralanmıştır;

1. Web Servisleri: ASP.NET Web API veya ASP.NET Core Web API ile RESTful servisler, oluşturarak veri alışverişi yapılabilir.
2. gRPC: Google tarafından geliştirilen hızlı ve verimli bir Remote Procedure Call (RPC) protokolüdür. Grpc.AspNetCore ile kolayca kullanılabilir. Performans açısından REST API'lere göre genelde daha hızlıdır.
3. Message Queue: Uygulamalar arasında mesaj geçişi ile asenkron haberleşme sağlar. RabbitMQ, Apache Kafka, ActiveMQ ve Azure Service tercih edilebilir.
4. SignalR: Sohbet gibi gerçek zamanlı iletişim için kullanılır. Sunucudan istemcilere anlık veri gönderebilir.

Bir ağ söz onuşu olduğunda aşağıdaki kavramların üzerinden geçmek gerekir;

Her bir bilgisayarlar ağ üzerinde bir Internet Protocol (IP) adresine sahiptir. Bu adresleri ezberlememek için bilgisayarın IP adresi yerine Uniform Resource Locator (URL) adı verilen bir dizgi (string) metni kullanılır. Bu URL ile IP eşleştirmesini alan adı sunucuları (name server) yapar.

```
http://www.ornek.org/
```

Bilgisayarlardaki her bir kaynak da URL olarak gösterilebilir.

```
http://www.ornek.org/index.html
http://www.ornek.org/CSS/index.css
http://www.ornek.org/Images/logo.jpg
```

Portlar uygulamaların bilgisayarda çalıştığını gösteren sanal kapılardır. Bilgisayarlar genellikle tek bir bağlantı (single connection) ile ağa bağlanırlar. Buradaki tek bağlantıya sahip bilgisayara ağ üzerinden gelen paketlerin, hangi uygulamaya ait olduğunun belirlenmesi ve söz konusu paketin ilgili uygulamaya ulaştırılmasının sağlanması amacıyla portlar kullanılır.

```
http://www.ornek.org:80/
http://www.ornek.org:8080/
ftp://ftp.ornek.com:21/
```

Portlar 0 ile 65535 arasında olup, ilk 1024 tanesi özel uygulamalar için ayrılmıştır. Örneğin 80 numaralı port, Hyper Text Transmission Protocol (HTTP) sunucusunu, 25 numaralı port Simple Mail Transport Protocol (SMTP) sunucusunu ve 21 numaralı port ise File Transfer Protocol (FTP) sunucusunu temsil eder. URL'nin başındaki http ve ftp ise bilgisayara ulaşılan port üzerinde çalışan protokolü göstermektedir.

Web Servisler

Web servis, internet üzerinden uygulamalar arasında veri alışverişi yapmayı sağlayan bir yapıdır. C# ile web servis oluşturmanın en yaygın yolu ASP.NET Web API veya ASP.NET Core Web API kullanmaktır. Günümüzde ASP.NET Core Web API, modern, platformlar arası (cross-platform) ve performanslı olduğu için tercih edilir.

Representational State Transfer (REST), 2000 yılında Roy Fielding'in doktora tezinde tanımlanan bir mimari stildir. Web sunucuları ile tarayıcılar arasında kullanılan HTTP yöntemlerinin http için belirlenen görevleri dışında veri değişiminde kullanılması esasına dayanır.

REST kurallarına uygun bir şekilde tasarlanmış web servisine RESTful API denir. Bu API, kaynaklara (veri nesneleri) HTTP yöntemleri (GET, POST, PUT, DELETE vb.) üzerinden erişim sağlar. RESTful servisindeki her şey bir kaynak (resource) olarak görülür. Kaynaklar genelde isimlendirilmiş Uniform Resource Identifier (URI) ile tanımlanır. URI, web adresine benzer şekilde bir kaynağı eşsiz ve açık bir biçimde tanımlayan standart biçime sahip bir dizgidir (string).

<https://api.ornek.com/ogrenciler/3>

RESTful servisleri http yöntemlerini yeni görev vererek anlamlı kullanır;

Yöntem	Görevi	Örnek URI	Açıklama
GET	Veri getir	/ogrenciler	Tüm öğrencileri getir
GET	Belirli veriyi getir	/ogrenciler/5	ID'si 5 olan öğrenciyi getir
POST	Yeni veri oluştur	/ogrenciler	Yeni öğrenci ekle
PUT	Veriyi tamamen güncelle	/ogrenciler/5	Öğrenci güncelle
PATCH	Veriyi kısmen güncelle	/ogrenciler/5	Sadece postasını değiştir
DELETE	Veriyi sil	/ogrenciler/5	Öğrenciyi sil

Tablo 22. http Yöntemlerinin RESTful Serviste Kullanılması

RESTful servislerde her HTTP isteği bağımsızdır yani **durumsuzdur** (stateless);

- Sunucu önceki istekleri hatırlamaz. Durum bilgisi (örneğin, oturum bilgisi) her istekte genellikle JWT veya **jeton** (token) ile yeniden iletilmelidir.
- RESTful API isteklerinde taşınan veriler ön belleğe alınabilir. Web sunucusuna, verinin ne kadar süreyle geçerli olduğunu belirtmelidir.
- İstemci, ara katman olup olmadığını bilmez bu nedenle katmanlı mimari için uygundur. API istekleri Load Balancer, API Gateway gibi yönlendirici sunuculardan geçebilir.
- Veri alışverişi genellikle **JavaScript Object Notation** (JSON) tercih edilmekte birlikte XML gibi formatlarla da kullanılır.

Aşağıda JSON formatında örnek bir veri verilmiştir;

```
{
  "id": 1,
  "adi": "İlhan ÖZKAN",
  "posta": "ilhanozkan@outlook.com"
}
```

RESTful Web API Örneği

Örnek olarak oluşacak uygulamanın yapısı aşağıda verilmiştir;

WebAPI	→ Çözüm (Solution)
└─ WebApplication1	→ Proje (ASP.NET Core Web AP)
└─ Controllers	→ Denetçilerin Olacağı Klasör
└─ WeatherForecastController.cs	→ Class (WeatherForecastController-Controller)
└─ KitapKontroller.cs	→ Class (KitapController-Controller)
└─ WeatherForecas.cs	→ Class (WeatherForecast-Model)
└─ Program.cs	→ Ana (Main) Program

Projeyi oluşturmak için;

1. Visual Studio üzerinde *File→New→Project...* seçilerek **WebAPI** adlı çözüm içinde **WebApplication1** adlı projemiz **ASP.NET Core Web API** olarak oluşturulur.
2. Oluşan projede **Program.cs** ve **WeatherForecast.cs** ile **Controllers** klasöründe bulunan **WeatherForecastController.cs** dosyaları otomatik olarak oluşur.
3. İlk önce **WeatherForecast.cs** dosyasını inceleyelim;

```
namespace WebApplication1
{
    public class WeatherForecast
    {
        public DateOnly Date { get; set; }

        public int TemperatureC { get; set; }

        public int TemperatureF => 32 + (int)(TemperatureC / 0.5556);

        public string? Summary { get; set; }
    }
}
```

```
}
```

4. Burada sözü geçen **Controllers** dizini altındaki **WeatherController.cs** dosyası MVC mimari desenindeki denetçidir. **WeatherForecastController.cs** dosyası incelendiğinde URL **rotası** (route) üzerinden GET isteği ile gelindiğinde 5 adet rastgele hava tahmini üreteceği görülmektedir.

```
using Microsoft.AspNetCore.Mvc;

namespace WebApplication1.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class WeatherForecastController : ControllerBase
    {
        private static readonly string[] Summaries = new[]
        {
            "Freezing", "Bracing", "Chilly", "Cool", "Mild",
            "Warm", "Balmy", "Hot", "Sweltering", "Scorching"
        };

        private readonly ILogger<WeatherForecastController> _logger;

        public WeatherForecastController(ILogger<WeatherForecastController> logger)
        {
            _logger = logger;
        }

        [HttpGet(Name = "GetWeatherForecast")]
        public IEnumerable<WeatherForecast> Get()
        {
            return Enumerable.Range(1, 5).Select(index =>
                new WeatherForecast
                {
                    Date = DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
                    TemperatureC = Random.Shared.Next(-20, 55),
                    Summary = Summaries[Random.Shared.Next(Summaries.Length)]
                }).ToArray();
        }
    }
}
```

5. Son olarak **Program.cs** dosyasını inceleyelim. **builder.Services.AddControllers();** talimatı ile MVC deseninde biz nasıl **MVCBaşlat()** yöntemi var ise burada da bir web uygulamasının denetçileri web uygulamamıza **Controllers** klasöründen eklenir. Geri kalan tüm işlemler web sunucusunu http isteklerini ayarlamak için ek ayarlardır.

```
var builder = WebApplication.CreateBuilder(args);
/*
Controller tabanlı API oluşturmak için gerekli servisi aşağıdaki talimat kaydeder.
*/
builder.Services.AddControllers();
/*
Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
*/
builder.Services.AddOpenApi();

var app = builder.Build();
// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}
```

```

/*
HTTP isteklerini otomatik olarak HTTPS'e yönlendirmek için aşağıdaki talimat yazılır.
*/
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.Run();

```

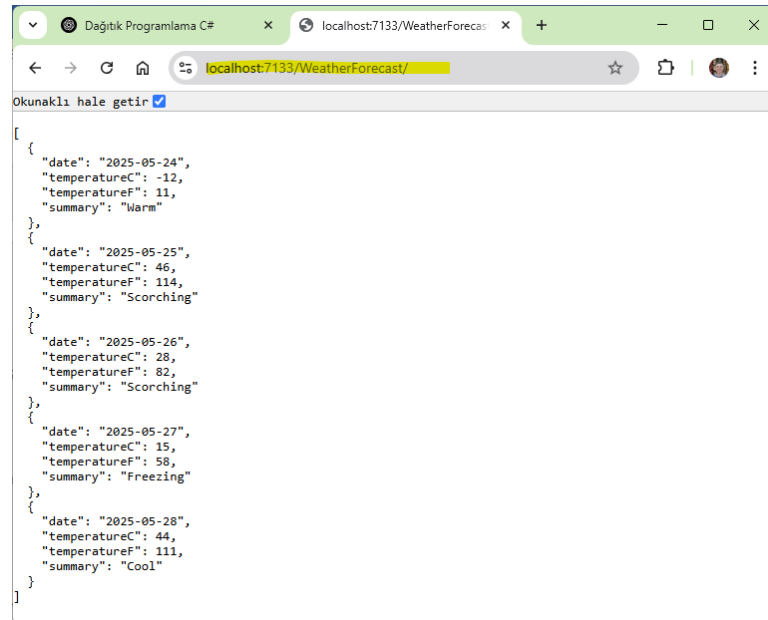
6. Programı çalıştırdığımızda API uygulamamız http web sunucu üzerinden istekleri kabul etmeye başlar.

```

info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:7133
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5085
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\CSharp\Samples\services\Web API\WebApplication1

```

7. Çalışan Web API üzerinden veri almak için bir web tarayıcısı açıp kullanabiliriz;



Şekil 66. WebApplication1 API Uygulamamızdan GET yöntemi ile Veri Çekme

8. Şimdi uygulamayı durdurup yeni bir denetçi ekleyelim. Burada model kullanmayacağız; Bunun için Solution Explorer üzerinde **Controllers** sağ tıklanarak *Add* → *Controller...* seçilir ve **KitapController.cs** adında *MVC Controller -Empty* denetçi oluşturulur.

```

using Microsoft.AspNetCore.Mvc;

namespace WebApplication1.Controllers
{
    public class KitapController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }
    }
}

```

9. Burada model ve gösterim ihtiyacımız olmadığından **Controllers/KitapController.cs** içinde aşağıdaki değişiklikleri yapalım;

```
using Microsoft.AspNetCore.Mvc;

namespace WebApplication1.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class KitapController : Controller
    {
        [HttpGet]
        public IActionResult TumKitaplariGetir()
        {
            var kitaplar = new[]
            {
                new { Id = 1, Ad = "Sefiller", Yazar = "Victor Hugo" },
                new { Id = 2, Ad = "Suç ve Ceza", Yazar = "Dostoyevski" }
            };

            return Ok(kitaplar);
        }

        [HttpGet("{id}")]
        public IActionResult KitapGetir(int id)
        {
            if (id == 1)
                return Ok(new { Id = 1, Ad = "Sefiller", Yazar = "Victor Hugo" });
            else
                return NotFound("Kitap bulunamadı");
        }
    }
}
```

10. Uygulamayı tekrar çalıştırıp tarayıcıda tüm kitapları getirmek için <https://localhost:7133/api/kitap> veya 1 numaralı kitabı getirmek için <https://localhost:7133/api/kitap/1> adresi yazılır.

API'mizden veri almak için örnekteki gibi tarayıcı kullanabiliriz ama bu her zaman yeterli olmaz. API'mizi test etmek için en yaygın HTTP test aracı olan Postman, komut satırından istek gönderme için cURL veya otomatik API dokümantasyonu da çıkaran Swagger UI kullanılabilir.

Örnek Kurumsal Uygulama

Kurumlar, içinde oldukları zamana bağlı olarak, iş yapma şekillerini ya da süreçlerini değiştirirler. Buna kullandıkları girdilerdeki değişim, istenilen çıktılardaki değişim ya da yasal mevzuattaki değişim sebep olmaktadır. Buna paralel olarak organizasyonu modelleyen ya da benzetim yapan yazılımların bu değişiklikleri karşılayabilir olması gerekir.

Yazılımlar, değişim olmadığı takdirde, sonsuza kadar çalışabilirler. Ancak yazılımda yapılacak her değişimin, canlı bir varlığa atılan bir bıçak darbesi olacağı akıldan çıkarılmamalıdır. Bu nedenle yazılımlar geliştirilirken, değişim yönetiminin ya da hangi sürecin ne kadar değişebileceğinin önceden belirlenmesi gerekir. Değişim yönetimi ne kadar başarılı olur ise yazılım da o kadar başarılı olacaktır.

Değişim yönetiminin yanında, yazılımda daha önceden yapılmış ve başarı sağlamış olan modellerin kullanılması da oldukça önemlidir. Yani yazılım tasarımı ve analizi yapılırken başarılı tecrübeleri içeren **desenlerin (pattern)** kullanılması gerekir. Bir yazılım, ne kadar fazla desenlere uygun yazılır ise, o kadar değişime açık olacaktır.

Bütün bunların yanında yazılımların katmanlara ayrılması işletim ve bakım açısından çok önemlidir. Çok katmanlı mimaride yazılan yazılımlar, ölçeklenebilirliği arttırdığı gibi birden fazla donanıma yükü paylaştırdığı için, küçük fiyatlı donanımlarla büyük işler başarılacaktır.

Örnek uygulama olarak müşteriden sipariş alan basit bir uygulama ele alınacaktır. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

ÇokKatmanlıUygulama	→ Çözüm (Solution)
└─ Entities	→ Proje (Class Library)
└─ Sipariş.cs	→ Class
└─ DataAccess	→ Proje (Class Library)
└─ SiparişRepository.cs	→ Class
└─ Business	→ Proje (Class Library)
└─ SiparişYöneticisi.cs	→ Class
└─ UI	→ Proje (Console Application)
└─ Program.cs	→ Ana (Main) Program

Projeyi oluşturmak için;

1. Visual Studio üzerinde *File→New→Project...* seçilerek **ÇokKatmanlıUygulama** adlı çözüm içinde **Entities** adlı ilk projemiz *Class Library* olarak oluşturulur.
2. Proje ile oluşan **Class1.cs** dosyasını **Sipariş.cs** olarak değiştiriyoruz ve aşağıdaki şekilde değiştiriyoruz;

```
//İş Varlığı Bileşenleri (BEC)
//Katmanlar arası taşınacak veri yapısını temsil eder
namespace Entities
{
    public class Sipariş
    {
        public Guid Id { get; set; }
        public string MüşteriAdı { get; set; } = string.Empty;
        public DateTime SiparişZamanı { get; set; }
    }
}
```

3. Daha sonra Solution Explorer üzerinde çözüme sağ tıklanarak *Add→New Project...* seçilerek **DataAccess** adlı ikinci projemiz *Class Library* olarak oluşturulur.
4. Solution Explorer üzerinde projemize sağ tıklanarak *Manage Nuget Packages...* tıklanır ve **Microsoft.Data.Sqlite Sqlite** paketi yüklenir.
5. Solution Explorer üzerinde **Entities** projesi sağ tıklanarak *Build* seçilir ve derlenir.
6. Solution Explorer üzerinde bu proje sağ tıklanarak *Add→Project Reference* ile **Entities** projesi dahil edilir.
7. Verileri veri tabanında işleyeceğimizden ilk önce **Siparişler** tablosunu oluşturmamız gerekir. Bunun için veri tabanı yöneticisinde aşağıdaki SQL komutu çalıştırılabilir;

```
CREATE TABLE Orders (
    Id TEXT PRIMARY KEY, -- GUID string olarak
    CustomerName TEXT NOT NULL,
    OrderDate TEXT NOT NULL -- ISO 8601 tarih formatında saklanabilir
);
```

8. Daha sonra bu tabloyu veri eklemek ve verileri okumak için kullanabiliriz. Proje ile oluşan **Class1.cs** dosyasını **SiparişRepository.cs** olarak değiştiriyoruz ve aşağıdaki şekilde değiştirilir;

```
//Veri Erişim Bileşenleri (DAC)
//Veri kaynağında CRUD işlemlerini yapar
using Entities;
using Microsoft.Data.Sqlite;

namespace DataAccess
{
    public class SiparişRepository
    {
        private readonly string connectionString;
        public SiparişRepository(string connectionString)
        {
```



```

        this.connectionString = connectionString;
    }
    public List<Sipariş> GetAll()
    {
        var orders = new List<Sipariş>();
        using var conn = new SqliteConnection(connectionString);
        conn.Open();
        using var cmd = conn.CreateCommand();
        cmd.CommandText="SELECT Id, MüşteriAdı, SiparişZamanı FROM Siparişler";

        using var reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            orders.Add(new Sipariş
            {
                Id = reader.GetGuid(0),
                MüşteriAdı = reader.GetString(1),
                SiparişZamanı = reader.GetDateTime(2)
            });
        }
        conn.Close();
        return orders;
    }
    public void Add(Sipariş sipariş)
    {
        using var conn = new SqliteConnection(connectionString);
        conn.Open();
        using var cmd = conn.CreateCommand();
        cmd.CommandText = "INSERT INTO Siparişler (Id, MüşteriAdı, SiparişZamanı)" +
            "VALUES(@Id, @Ad, @Zaman)";
        cmd.Parameters.AddWithValue("@Id", sipariş.Id);
        cmd.Parameters.AddWithValue("@Ad", sipariş.MüşteriAdı);
        cmd.Parameters.AddWithValue("@Zaman", sipariş.SiparişZamanı);
        cmd.ExecuteNonQuery();
        conn.Close();
    }
}
}

```

9. Ardından Solution Explorer üzerinde çözüme sağ tıklanarak *Add→New Project...* seçilerek **Business** adlı üçüncü projemizi *Class Library* olarak oluşturuyoruz.
10. Bu projeye de sağ tıklanarak *Add→Project Reference* ile **Entities** projesi dahil edilir.
11. Solution Explorer üzerinde **DataAccess** projesi sağ tıklanarak *Build* seçilir ve derlenir.
12. **Business** projesi sağ tıklanarak *Add→Project Reference* ile **DataAccess** projesi dahil edilir.
13. Proje ile oluşan **Class1.cs** dosyasını **SiparişYöneticisi.cs** olarak değiştiriyoruz ve aşağıdaki şekilde değiştirilir;

```

//İş Mantığı Bileşenleri (BLC)
//İş kurallarını uygular, doğrulama (validaton) yapar
using Entities;
using DataAccess;
namespace Business
{
    public class SiparişYöneticisi
    {
        private readonly SiparişRepository depo;
        public SiparişYöneticisi(SiparişRepository repo)
        {
            depo = repo;
        }
        public List<Sipariş> GetSiparişler()
    }
}

```

```

    {
        return depo.GetAll();
    }
    public void SiparişOluştur(string müşteriAdı)
    {
        if (string.IsNullOrEmpty(müşteriAdı))
            throw new ArgumentException("Müşteri adı gereklidir");
        var yeniSipariş = new Sipariş
        {
            Id = Guid.NewGuid(),
            MüşteriAdı = müşteriAdı,
            SiparişZamanı = DateTime.UtcNow
        };
        depo.Add(yeniSipariş);
    }
}

```

14. Son olarak Solution Explorer üzerinde çözüme sağ tıklanarak *Add→New Project...* seçilerek **UI** adlı dördüncü projemizi *Console Application* olarak oluşturuyoruz.
15. Bu proje sağ tıklanarak *Add→Project Reference...* ile **DataAccess** projesi dahil edilir.
16. Solution Explorer üzerinde **Business** projesi sağ tıklanarak *Build* seçilir ve derlenir.
17. **UI** projesine sağ tıklanarak *Add→Project Reference...* ile **Business** projesi dahil edilir.
18. Proje ile oluşan **Program.cs** aşağıdaki şekilde değiştirilir;

```

// See https://aka.ms/new-console-template for more information
//Kullanıcı Arayüçü (UI-Console Application)
//Kullanıcı ile etkileşim sağlar

using Business;
using DataAccess;

var connectionString = @"Data Source=D:\CSharp.sqlite";
var depo = new SiparişRepository(connectionString);
var yönetici = new SiparişYöneticisi(depo);

// Yeni sipariş ekleme
Console.WriteLine("Müşteri adı girin: ");
string ad = Console.ReadLine() ?? "";
yönetici.SiparişOluştur(ad);
Console.WriteLine("Sipariş eklendi!");

// Siparişleri listeleme
var siparişler = yönetici.GetSiparişler();
foreach (var sipariş in siparişler)
{
    Console.WriteLine($"{sipariş.Id} - {sipariş.MüşteriAdı} - {sipariş.SiparişZamanı}");
}

```

19. Uygulamayı çalıştırmak için Solution Explorer üzerinde **UI** projesi sağ tıklanarak *Set As Startup Project* seçilir. Artık programımız çalışır.